

Performance testing on MQ SaaS AWS Reserved Instance

Objective

In our previous AWS report, data was published from scenarios running in a manually deployed OpenShift environment on EC2 hardware. Whilst this allows full control of your messaging environment, many customers are looking to run their messaging workload as a managed service. This paper will look at the performance of running Queue Managers(QM) within IBM MQ SaaS Reserved Instance on AWS (MQ SaaS).

MQ SaaS QM are deployed using Native HA, with data replicated across three Availability Zones(AZ) to provide high availability and resilience.

4 sizes of QM are available, Extra Small, Small, Medium and Large. Each tier provides increasing levels of connection capacity, compute resources, and storage capability. This report will demonstrate the capabilities of each size to help guide you in selecting the correct tier for your workload.

The test scenario used is the Request/Responder scenario (RR-CC) as featured in our xLinux MQ Performance report. Each round trip is comprised of two message Puts and two message Gets and utilizes pairs of queues to communicate between the requester clients and responder applications.

Queue Manager size comparison table

Metric / Tier	XSmall	Small	Medium	Large
Virtual Processor Cores (VPC)	0.5	1	2	4
Concurrent connections (maximum)	30	50	300	1,000
Memory (RAM)	1 GB	1 GB	2 GB	4 GB
Disk size	20 GB	20 GB	40 GB	40 GB

Environment

Config

MQ: V10.0.0r2

Worker nodes: 3

Storage specification: EBS GP3

Test client: cphtestp (containerized MQI test application, ran in same AZ to active QM)

Test client config: 4 CPU request, 8 CPU limit, 8GB RAM request/limit, pinned to us-east-1c

Client location: Same AZ as active QM

Application: Private application endpoints were used for application connections

Persistence: Non Persistent and Persistent messaging results

Msg Size: 2KB/20KB/200KB

QM Type: NHA QM are deployed by default by MQ SaaS

TLS: TLS13 is enabled for all application workload and HA group replication

JSON: JSON payloads used by cphtestp

SHARECNV: A sharecnv value of 1 was used to utilise individual sockets for each client connection
IMGLOGLN: Defaults to 25% of available PV. i.e. 5GB for XSMALL, SMALL, 10GB for MEDIUM, LARGE

Network

The measured latency between the Availability Zones hosting the active and replica queue managers was less than 1 ms.

A single threaded iperf3 test between nodes in different Availability Zones(AZ) measured ~5Gbps.

Storage

EBS GP3 storage is utilized for QM storage (and is not configurable). 20GB volumes are created for XSMALL and SMALL sizes, 40GB volumes are created for MEDIUM and LARGE sizes.

GP3 Throughput: 128MB/s

GP3 IOPS: 3000

Encrypted: true

XSmall QM Test results

This section contains the results from the NP testing against the XSmall MQ SaaS QM:

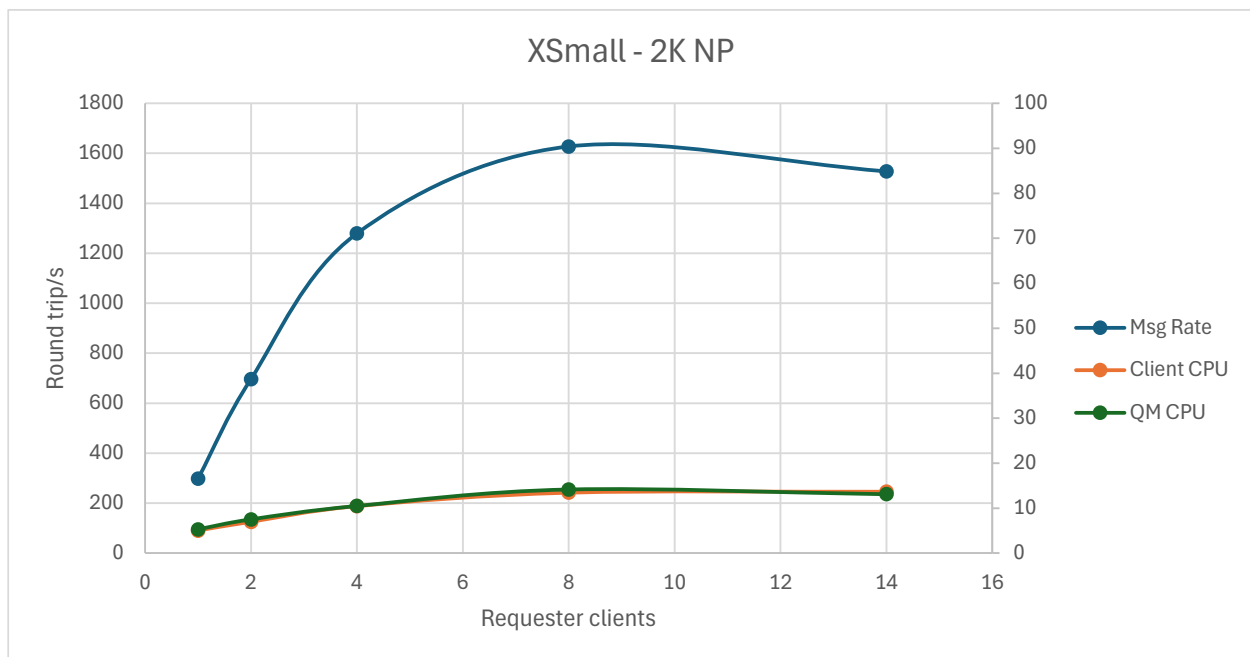


Figure 1 - 2K Non Persistent messaging

In the chart above for 2K Non Persistent messaging, the throughput of the XSmall QM achieves over 1600 round trip/s with 8 requester clients.

The data table below provides additional data for the 20K and 200K message sizes:

Tier	MsgSize	Persistence	Peak Msg Rate (Round trip/s)	QM CPU	Estimated IO (MB/s)
XSmall	2K	NP	1628	14	N/A
XSmall	20K	NP	1165	14	N/A
XSmall	200K	NP	366	14	N/A

Table 1 - XSmall Non Persistent Messaging Results

The following chart shows the performance of 2K Persistent messaging with the XSmall QM:

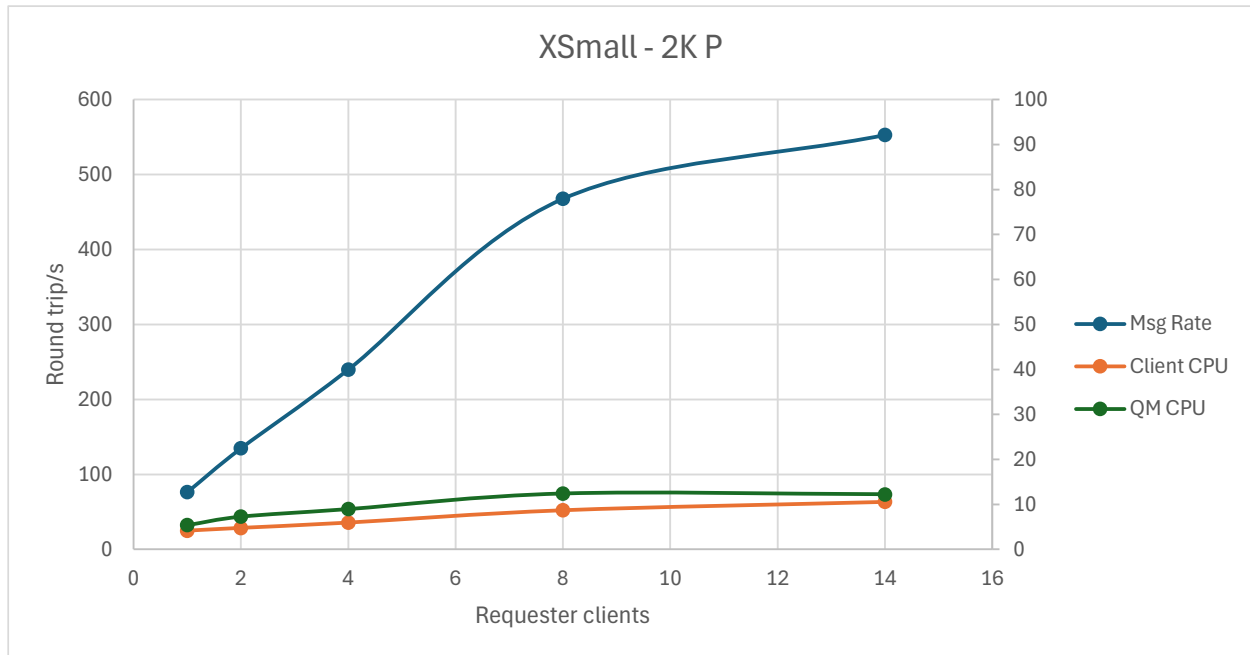


Figure 2 - 2K Persistent messaging

The chart above shows that with Persistent messages, the XSmall QM can process over 500 round trip/s at 14 requester clients.

The data table below provides additional data for the 20K and 200K message sizes:

Tier	MsgSize	Persistence	Peak Msg Rate (Round trip/s)	QM CPU	Estimated IO (MB/s)
XSmall	2K	P	552	12	3
XSmall	20K	P	374	13	15
XSmall	200K	P	119	13	47

Table 2 - XSmall Persistent Messaging Results

The XSmall QM data shows good overall performance in AWS for the lowest tier offered by MQ SaaS. Those looking to use larger numbers of connections, increased throughput or IO capacity will require one of the larger tier sizes.

Small QM Test results

This section contains the results from the NP testing against the Small MQ SaaS QM:

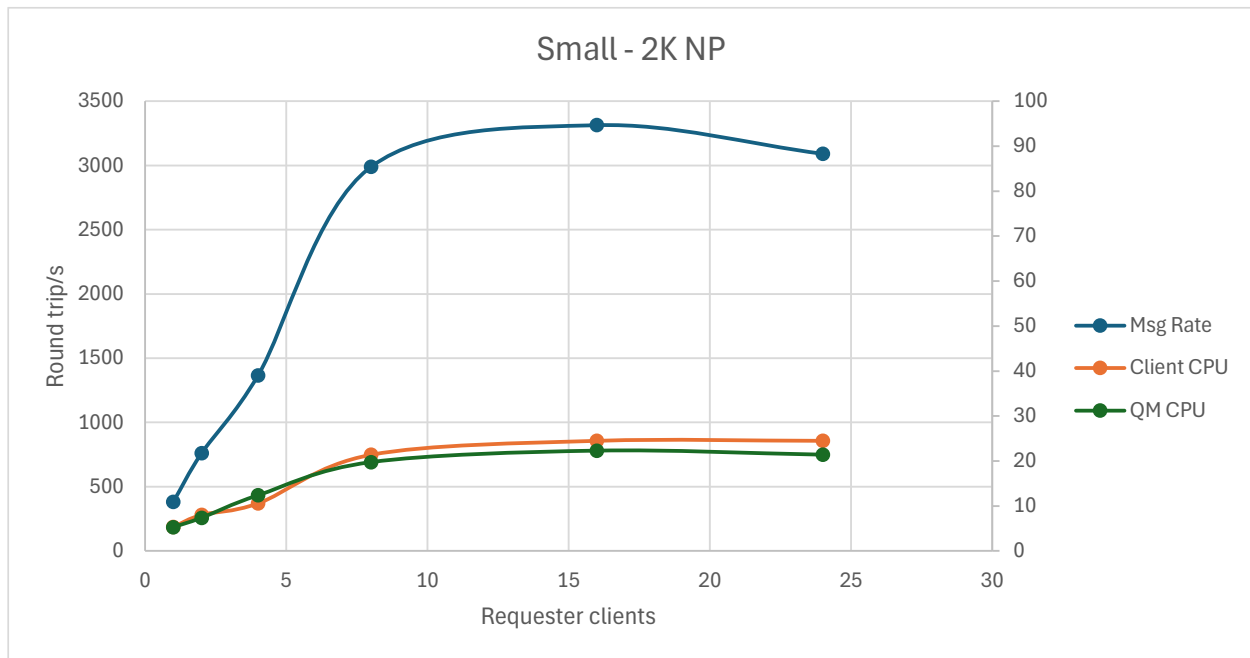


Figure 3 - 2K Non Persistent messaging

In the chart above for 2K Non Persistent messaging, the throughput of the Small QM achieves over 3300 round trip/s with 16 requester clients.

The data table below provides additional data for the 20K and 200K message sizes:

Tier	MsgSize	Persistence	Peak Msg Rate (Round trip/s)	QM CPU	Estimated IO (MB/s)
Small	2K	NP	3314	22	N/A
Small	20K	NP	2351	24	N/A
Small	200K	NP	680	24	N/A

Table 3 - Small Non Persistent Messaging Results

The following chart shows the performance of 2K Persistent messaging with the Small QM:



Figure 4 - 2K Persistent messaging

The chart above shows that with Persistent messages, the Small QM can process over 1100 round trip/s at 24 requester clients.

The data table below provides additional data for the 20K and 200K message sizes:

Tier	MsgSize	Persistence	Peak Msg Rate (Round trip/s)	QM CPU	Estimated IO (MB/s)
Small	2K	P	1185	21	7
Small	20K	P	826	22	34
Small	200K	P	262	22	103

Table 4 - Small Persistent Messaging Results

The Small QM data shows good scaling performance in AWS when compared to the lowest tier offered by MQ SaaS, with all scenarios demonstrating approximately twice the messaging rate. This is a great starting point for any new MQ deployment with larger tiers available for more demanding workloads.

Medium QM Test results

This section contains the results from the NP testing against the Medium MQ SaaS QM:



Figure 5 - 2K Non Persistent messaging

In the chart above for 2K Non Persistent messaging, the throughput of the Medium QM achieves over 5000 round trip/s with 16 requester clients.

The data table below provides additional data for the 20K and 200K message sizes:

Tier	MsgSize	Persistence	Peak Msg Rate (Round trip/s)	QM CPU	Estimated IO (MB/s)
Medium	2K	NP	5341	35	N/A
Medium	20K	NP	4050	43	N/A
Medium	200K	NP	1152	46	N/A

Table 5 - Medium Non Persistent Messaging Results

The following chart shows the performance of 2K Persistent messaging with the Medium QM:

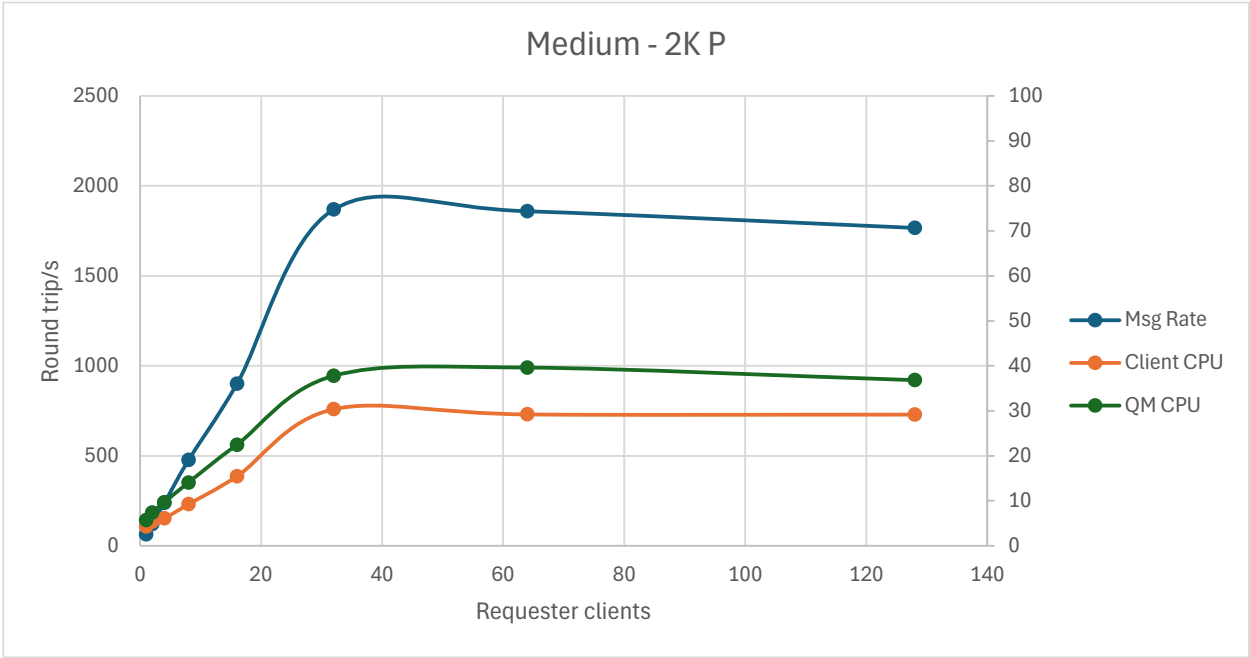


Figure 6 - 2K Persistent messaging

The chart above shows that with Persistent messages, the Medium QM can process over 1800 round trip/s at 32 requester clients.

The data table below provides additional data for the 20K and 200K message sizes:

Tier	MsgSize	Persistence	Peak Msg Rate (Round trip/s)	QM CPU	Estimated IO (MB/s)
Medium	2K	P	1869	38	11
Medium	20K	P	1361	40	56
Medium	200K	P	322	32	126

Table 6 - Medium Persistent Messaging Results

The Medium QM data shows good performance improvement over the Small QM and supports the demands of heavier messaging workloads.

Large QM Test results

This section contains the results from the NP testing against the Large MQ SaaS QM:

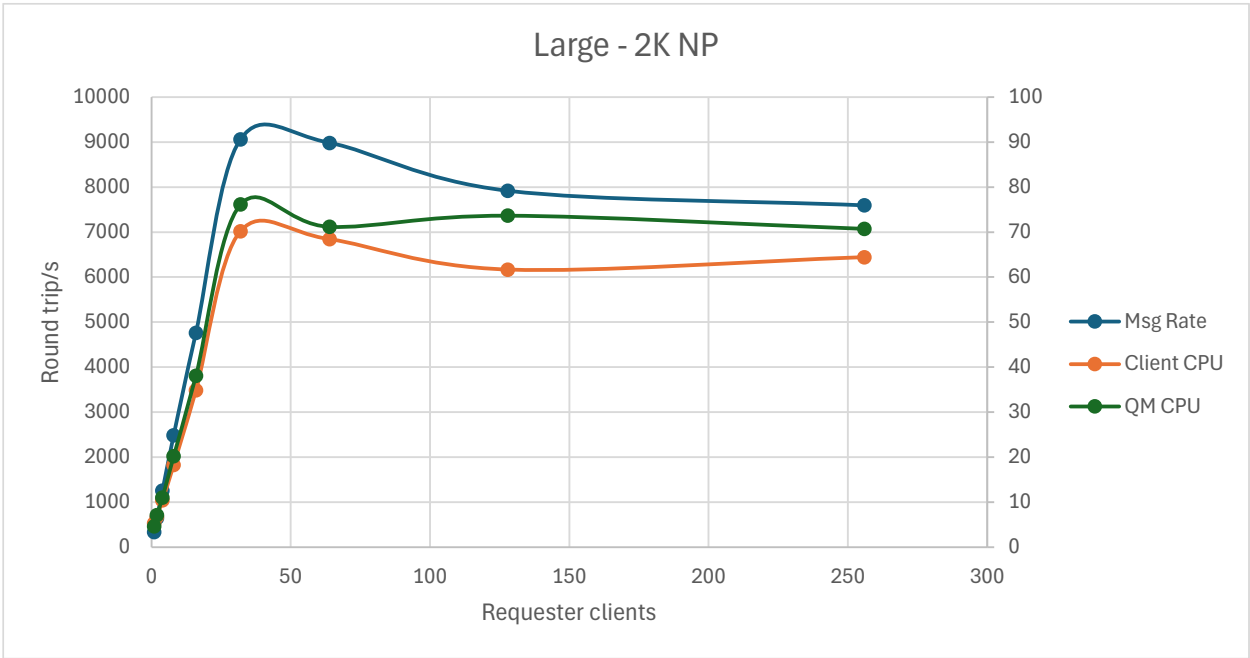


Figure 7 - 2K Non Persistent messaging

In the chart above for 2K Non Persistent messaging, the throughput of the Large QM achieves over 9000 round trip/s with 32 requester clients.

The data table below provides additional data for the 20K and 200K message sizes:

Tier	MsgSize	Persistence	Peak Msg Rate (Round trip/s)	QM CPU	Estimated IO (MB/s)
Large	2K	NP	9062	76	N/A
Large	20K	NP	5909	77	N/A
Large	200K	NP	1796	87	N/A

Table 7 – Large Non Persistent Messaging Results

The following chart shows the performance of 2K Persistent messaging with the Large QM:

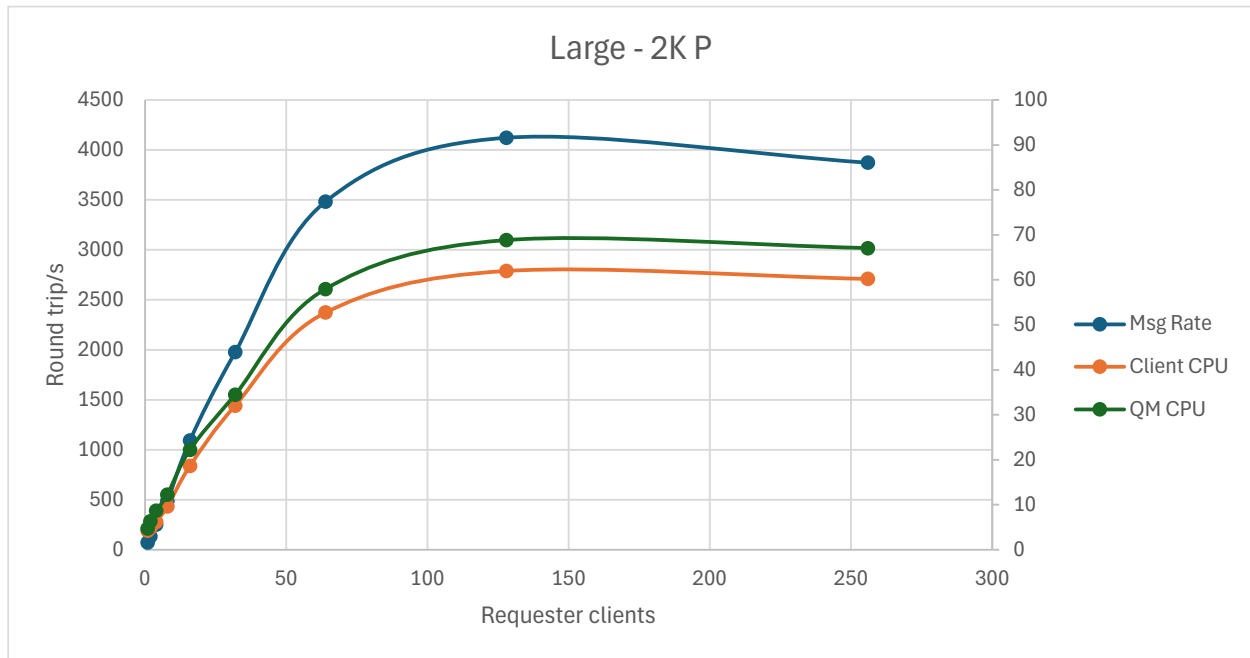


Figure 8 - 2K Persistent messaging

The chart above shows that with Persistent messages, the Large QM can process over 4000 round trip/s at 128 requester clients.

The data table below provides additional data for the 20K and 200K message sizes:

Tier	MsgSize	Persistence	Peak Msg Rate (Round trip/s)	QM CPU	Estimated IO (MB/s)
Large	2K	P	4123	69	24
Large	20K	P	2382	65	98
Large	200K	P	324	28	127

Table 8 - Large Persistent Messaging Results

The Large QM data shows good further improvement over the Medium QM and offers the highest messaging rates of the MQ SaaS tiers, supporting over 4000 round trip/s for Persistent messages and over 9000 round trip/s for Non Persistent messages.

Results Summary

The results presented so far have shown the performance levels of the individual QM tier. To help compare across the tiers, the data is shown below for the peak throughput across all tiers for each of the message sizes tested. Architects can use the data to guide them to the appropriate choice of tier for their deployment.

The graph below shows how the MQ SaaS Queue Managers perform with a 2K message size:

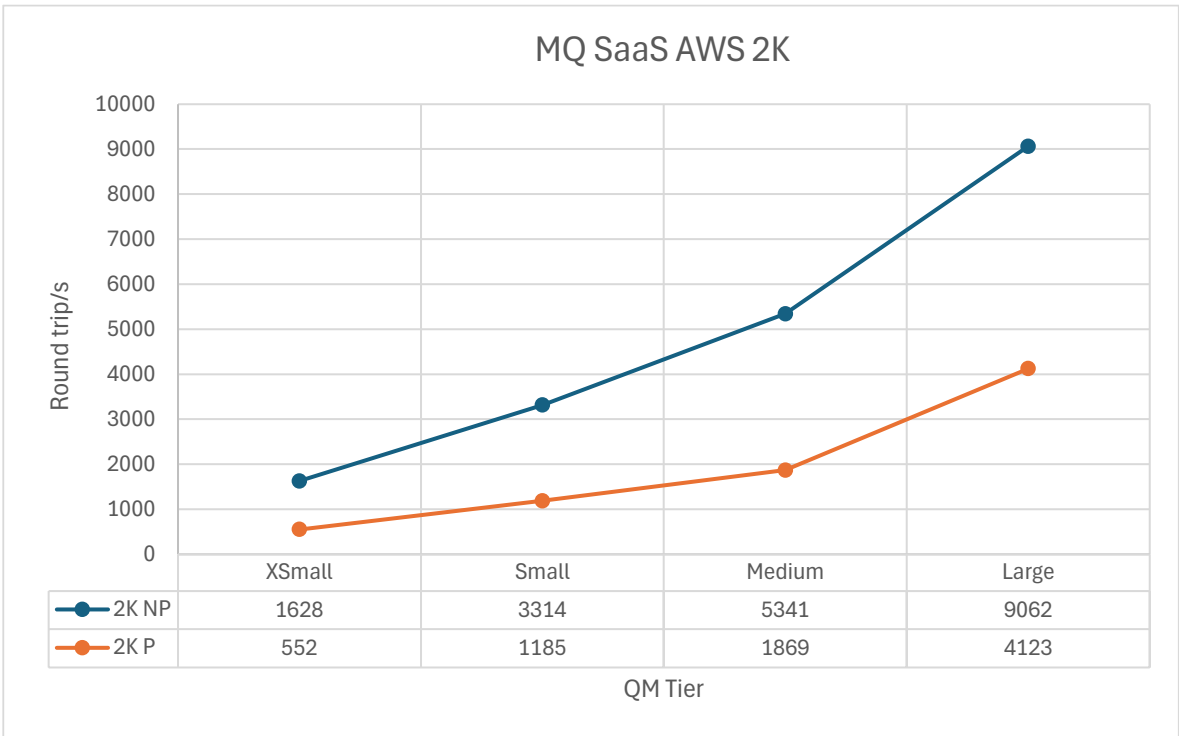


Figure 9 - 2K Scaling

Use the above to guide you to selecting the correct tier. Do note however that any Kubernetes environment isn't fully deterministic and sufficient processing headroom should be provided.

The graph below shows how the MQ SaaS Queue Managers perform with a 20K message size:

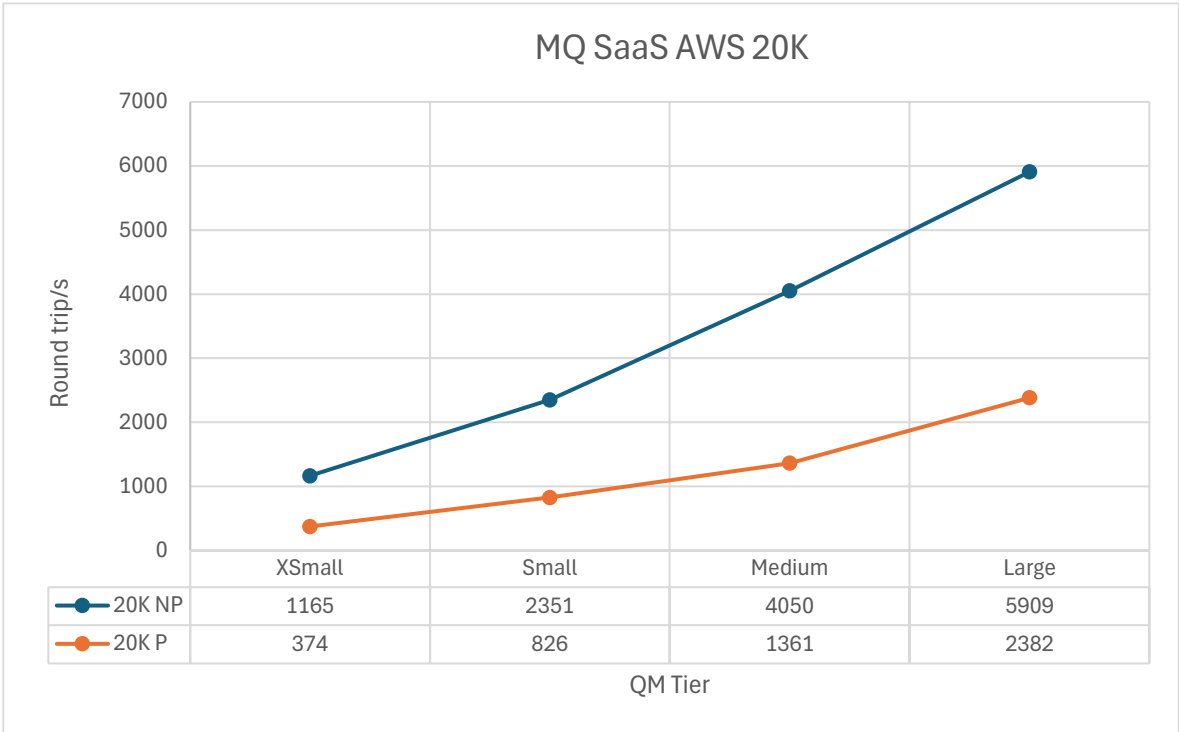


Figure 10 - 20K Scaling

The graph above shows good scaling across the tiers allowing the most appropriate tier to be selected for the intended workload.

The graph below shows how the MQ SaaS Queue Managers perform with a 200K message size:

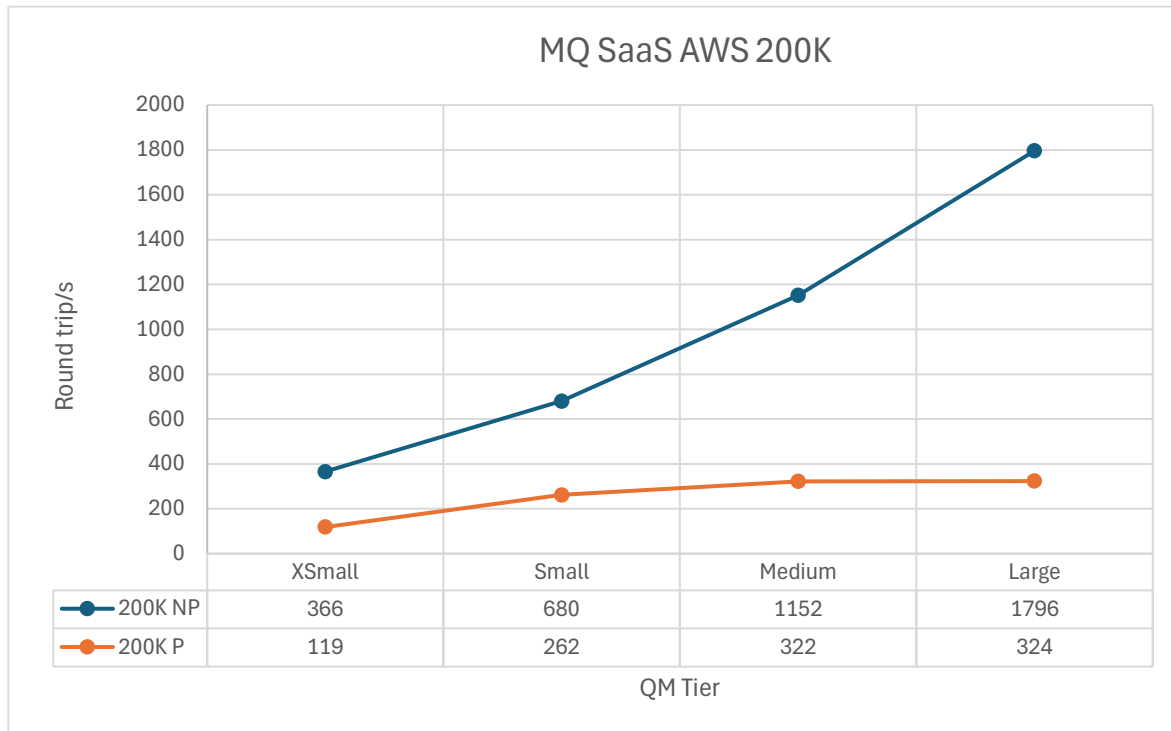


Figure 11 - 200K Scaling

The graph above shows that the Medium QM saturates the storage capabilities at the 200K message size when using Persistent messages. There is however spare capacity for Non Persistent workload.

Conclusion

The performance results presented in this whitepaper show that MQ SaaS Reserved Instance on AWS provides a robust platform for deploying Queue Managers on dedicated infrastructure. Across the test scenarios examined, the environment demonstrated the capability to support a diverse set of workload patterns and throughput requirements.

In addition to performance, the combination of highly available Queue Managers and rapid provisioning enables greater deployment flexibility and operational agility, supporting modern application architectures and evolving business needs.

As performance characteristics are influenced by application design, workload composition, message persistence requirements, and deployment configuration, customers should conduct validation testing using their own environments and usage patterns. The results presented in this paper should be used as guidance when selecting an MQ SaaS Reserved Instance tier, with workload-specific testing used to confirm performance objectives.

Links

cphtestp

<https://github.com/ibm-messaging/cphtestp>

Performance Test on NHA:CRR

<https://ibm-messaging.github.io/mqperf/PerformanceTestOnNHACRR.pdf>

Native HA Performance on OpenShift

<https://github.com/ibm-messaging/mqperf/blob/gh-pages/openshift/OCP4.12-NativeHA.pdf>

MQ on AWS (OpenShift on EC2 featuring SIQM, NativeHA and NHA:CRR)

<https://ibm-messaging.github.io/mqperf/NHACRRPerformanceTestAWS.pdf>

Monitoring system resource using amqsrua

<https://www.ibm.com/docs/en/ibm-mq/9.4?topic=stmat-monitoring-system-resource-usage-by-using-amqsrua-command>

Metrics published on the system topics

<https://www.ibm.com/docs/en/ibm-mq/9.4?topic=trace-metrics-published-system-topics>

MQ SaaS on AWS Announcement

<https://www.ibm.com/new/announcements/ibm-mq-saas-on-aws-messaging-built-for-hybrid-and-multi-cloud-connectivity>